MRP-FUZZ: Decoupling Input Generation and Thread-Interleaving Exploration for Linux Kernel Concurrency Fuzzing

Anonymous

***Abstract*—Detecting data races in the Linux kernel requires searching both the input space and thread-interleaving space. Existing kernel concurrency fuzzers couple these two searches by exploring interleavings during fuzzing. However, most inputs generated by these fuzzers have little potential to trigger real races, and scheduling interleavings for such inputs causes a significant waste of testing effort.**

In this paper, we propose MRP-FUZZ, a novel kernel concurrency fuzzing framework that decouples input generation and thread-interleaving exploration by introducing a new concept, May Race Pair (MRP). Unlike a real data race, whose variable access pair actually occurs concurrently, an MRP represents a conflicting variable access pair that occurs within a dynamically determined time threshold. During the fuzzing campaign, MRP-FUZZ performs pure input generation and collects triggered MRPs, which are used by our LLM-assisted semantic mutation to generate diverse syscall program groups. Then, MRP-FUZZ sets up a scheduling instance that consumes the collected MRPs and explores possible thread interleavings to confirm them as real races. Our scheduling integrates techniques including history-aware context recovery, context-aware interleaving scheduling, and candidate MRP prioritizing. We implement MRP-FUZZ for the Linux kernel and evaluate it on eight common modules in Linux 6.17. MRP-FUZZ finds 187 unique real data races, among which 59 are harmful. Compared to existing kernel concurrency fuzzers, MRP-FUZZ finds substantially more real races with more MRPs. We expect MRP-FUZZ to benefit kernel testing community by introducing a new concurrency fuzzing paradigm.

1. Introduction

Data races are a common and critical class of concurrency issues in the Linux kernel. A data race occurs when a conflicting access pair executes concurrently. A conflicting access pair is defined as two accesses from different threads to the same or overlapping memory range, where at least one access is a write. In the Linux kernel, data races can cause data corruption [1], memory corruption [2]–[4], and severe security vulnerabilities such as privilege escalation [5]–[7].

Detecting data races is more challenging than detecting single-threaded bugs, because race exposure depends on both suitable inputs and suitable schedules. The input dimension determines whether concurrently executed threads can create a conflicting access pair. The schedule dimension determines whether these accesses occur simultaneously. Therefore, detecting data races in the Linux kernel must consider both input and schedule dimensions. This extremely enlarges the search space, making data races difficult to detect in practice.

Many approaches have been proposed to detect data races in the Linux kernel. Coverage-guided fuzzers primarily explored the input dimension alone [8]–[17], relying on runtime data-race checkers such as KCSAN [18] and KTSAN [19] to identify races during execution. However, their main feedback signal is code coverage, which reflects sequential system executions but provides little guidance for exploring thread interleavings, and therefore limits their ability to expose data races.

Recent work has increasingly recognized that data-race detection must reason about both inputs and thread interleavings. One line of work first reduces the search space through static race analysis, memory-access traces collected from sequential executions, or learned predictors trained from prior executions, and then performs concurrent execution with strategic scheduling within the reduced space [20]–[22]. Another line of concurrency-aware fuzzing uses runtime concurrency information to guide thread-interleaving exploration, increasing the probability of triggering concurrency issues [23]–[28]. These approaches substantially improve practical race detection because they move beyond single-threaded coverage and explicitly consider schedule dimensions.

However, existing approaches remain inefficient because they still deeply couple their input generation and thread-interleaving exploration when detecting data races. Specifically, existing approaches have to repeatedly execute the inputs and insert scheduling points (*e.g.*, thread delay) during their executions. However, most of the inputs they randomly generate have limited potential to trigger real races, as a result of which existing approaches waste a significant amount of computing resources in scheduling these inputs. The root cause of this inefficiency is that input generation and thread scheduling are coupled. As they lack a systematic way to identify the inputs that have a higher potential to trigger data races under scheduling, existing approaches either have to schedule most inputs once they are generated, including many low-potential ones, or rely on concurrency feedback, such as covered thread interleavings, whose collection requires repeatedly scheduling for each generated input.

In this paper, we propose a new concurrency fuzzing paradigm, ***decoupling input generation and thread scheduling***, to significantly increase the efficiency of race detection in the Linux kernel. Thread scheduling is much more expensive than input generation, as scheduling typically requires repeated input executions or fine-grained system execution controls. Therefore, our insight is that the fuzzers should perform thread scheduling for only the inputs that have a higher potential to trigger real data races, instead of applying expensive scheduling to every generated input. Based on this insight, we separate data race detection into two decoupled processes: (1) generating inputs and selecting the ones that deserve to be scheduled; (2) scheduling the execution of the given input and determining whether it indeed can trigger a data race.

The core part of such decoupling is how to determine whether a given input has the potential to trigger real races and thus deserves the expensive scheduling. To address it, we introduce a new concept, ***May Race Pair*** (MRP). An MRP represents a conflicting access pair that occurs within a dynamically determined time threshold. It does not prove the existence of a data race; rather, it indicates a scheduling-worthy pair: when two accesses target the same memory address and naturally occur within a short time window, they are more likely to occur concurrently under scheduling than other access pairs. With MRP, we can effectively select the inputs that have a higher potential to trigger data races and perform expensive scheduling on only these inputs. Moreover, an MRP records the specific conflicting access pair, which enables the scheduling for the input to focus on a pair of concrete code sites, significantly reducing the scheduling cost.

Based on this concept, we design MRP-FUZZ, a kernel concurrency fuzzing framework. MRP-FUZZ consists of two decoupled instances: input-generation instance and thread-scheduling instance. The two instances run in parallel. The input-generation instance produces inputs that trigger MRPs, while the thread-scheduling instance consumes these inputs by scheduling their thread execution to trigger the potential data races. To maximize the number of detected data races, the throughput of these instances should be the same, which motivates our design of the dynamically determined time threshold specified in the MRP. This threshold involves a tradeoff: (1) a smaller threshold admits only conflicting access pairs with shorter temporal distances, which are usually more promising for triggering real races under scheduling, but may produce too few candidate MRPs and underutilize the thread-scheduling instance; (2) a larger threshold allows the input-generation instance to collect more MRPs, but may also admit more low-potential candidates, which may waste the scheduling effort. MRP-FUZZ adopts a backpressure-guided control to dynamically adjust the threshold. Specifically, MRP-FUZZ decreases the time threshold when the collected MRPs accumulate faster than they can be consumed, so that the input-generation instance admits fewer MRPs with shorter temporal distances and the thread-scheduling instance can spend its limited budget on more promising candidates. Conversely, MRP-FUZZ increases the time threshold when the collected MRPs are insufficient, ensuring enough candidate MRPs to avoid starvation of the thread-scheduling instance.

The input-generation instance generates syscall groups. Each group consists of two sequences of syscalls. Given a syscall group, this instance executes the two syscall sequences concurrently without involving thread scheduling. The instance collects the syscall groups that trigger new MRPs. We propose LLM-assisted semantic mutation for generating more interesting syscall groups. Specifically, we provide the LLM with module-specific mutation constraints and cross-sequence semantic interaction objectives. Guided by this information, we leverage its reasoning capabilities and pretrained knowledge to understand and semantically mutate promising syscall groups selected based on MRP feedback.

The thread-scheduling instance consumes the collected syscall groups and schedules them to confirm whether they can indeed trigger data races. We integrate three methods to facilitate our scheduling: history-aware context recovery to recover the kernel state required to reproduce candidate MRPs, context-aware interleaving scheduling to distinguish different dynamic occurrences of the same static access site using finer-grained runtime contexts, and candidate MRP prioritizing to prioritize candidate inputs based on previous scheduling results.

We implemented MRP-FUZZ, whose source code is available at an anonymized artifact repository at https://anonymous.4open.science/r/artifact-1590-6093. We evaluated MRP-FUZZ on eight common Linux kernel modules in Linux 6.17. In total, it found 187 unique real-world data races, among them 59 are identified as harmful races. We reported the harmful races. As of paper submission, 39 bugs have been confirmed, and 15 of the bugs have been fixed. These races may result in critical security vulnerabilities like concurrent use-after-frees. The comparison results indicate that our approach can find significantly more data races than existing approaches including SegFuzz and CONZZER within the same testing budget.

In summary, we make the following contributions:

- At the conceptual level, we propose a new paradigm for kernel concurrency fuzzing: decoupling input generation and thread scheduling. We further introduce a new concept, May Race Pair (MRP), to facilitate the decoupling.
- At the technical level, we design MRP-FUZZ, a kernel concurrency fuzzing framework that realizes this paradigm. We integrate MRP-FUZZ with LLM-assisted semantic mutation to generate diverse system inputs, and with history-aware context recovery, context-aware interleaving scheduling, and candidate MRP prioritizing to efficiently explore promising thread interleavings.
- At the practical level, we implemented and evaluated MRP-FUZZ on eight common Linux kernel modules in Linux 6.17. MRP-FUZZ found 187 unique real-world data races (59 harmful, 39 confirmed, and 15 fixed), outperforming the existing approaches.

2. Background

2.1. Data Race

In this work, we identify a data race when the two endpoints of a conflicting access pair are observed to overlap in execution. A conflicting access pair consists of two accesses from different threads to the same or overlapping memory range, with at least one access being a write. Thus, triggering a data race requires an input that produces a concrete conflicting access pair and a schedule that brings its two endpoint accesses into monitored overlap in time.

2.2. Concurrent Execution in the Linux Kernel

System calls (syscalls) are the primary interface through which user-space programs drive kernel behavior. A syscall can create and manipulate kernel objects, establish resource dependencies, update subsystem state, and trigger different allocation, access, and cleanup paths. In kernel fuzzing, syscalls serve as the de facto input representation.

To systematically exercise interactions between different threads, a concurrency fuzzer executes multiple system-call sequences concurrently. These executions may interact through the same object instance, related resources, or shared subsystem state, and may consequently produce a concrete conflicting access pair.

These concurrent executions inherently introduce a large interleaving space. Different syscall sequences may reach different shared states and trigger diverse memory accesses. Moreover, even the same concurrently executed sequences can interleave in many ways because each syscall handler contains numerous internal operations, blocking points, and memory accesses. To prevent interleavings from corrupting shared kernel data, the kernel relies on synchronization mechanisms to enforce safe thread execution orders. However, missing or incorrectly scoped synchronization may allow conflicting accesses to occur concurrently, manifesting as data races.

Accordingly, kernel data-race detection involves both an input dimension and a schedule dimension. The input dimension determines whether the executed syscall sequences produce a concrete conflicting access pair, while the schedule dimension determines whether the two accesses temporally overlap. Without proper inputs to establish the required kernel states and execution contexts, scheduling cannot drive the thread execution to reach potentially conflicting code paths. Conversely, input generation alone cannot ensure that these reached accesses overlap in time.

3. Basic Idea

3.1. May Race Pair

We decouple the input generation and thread scheduling by selecting the inputs that have the potential to trigger data races and performing thread scheduling for only the selected inputs. In this case, the expensive costs caused by thread scheduling can be significantly reduced, and thus the overall efficiency of race detection can be improved. The research question in this design is: how to determine whether a given input deserves to be scheduled?

Kernel executions produce many cross-thread accesses to overlapping addresses, but not all of them indicate a meaningful interaction between two threads. For example, two accesses may overlap only because the same address has been released and later reused for a different kernel object, rather than because both accesses operate on the same live object. Moreover, even if the two accesses target the same kernel object, they may belong to entirely different lifecycle phases of the object, such as resource initialization and finalization, and thus are temporally isolated by a substantial time gap under normal execution. Therefore, we use temporal closeness as an additional admission criterion for conflicting access pairs: an observed conflicting access pair is interesting only when the two accesses occur within a bounded time window.

We propose a new concept, May Race Pair (MRP), which serves as the criterion for selecting interesting syscall groups. An MRP is an observed conflicting access pair whose two accesses occur within a short time threshold. The concrete value of such a time threshold is dynamically determined and is discussed in Section 3.3. An MRP is not a proof that the two accesses have already raced. Instead, it indicates that the syscall group has produced a memory-level conflict whose endpoints occur sufficiently close in time to warrant targeted interleaving exploration. During the concurrent execution of a syscall group, we record the runtime memory accesses. Each access is represented as

$$a = \langle n, s, tid, addr, size, type, ts \rangle,$$

where n identifies the static access site, s identifies the stack context, tid identifies the thread that performs the access, $addr$ and $size$ describe the accessed memory range, $type$ indicates whether the access is a read or a write, and ts records the access timestamp. Given two runtime accesses a_1 and a_2, we formalize a May Race Pair as

$$\begin{aligned} & p = \langle a_1, a_2, \Delta t \rangle, \quad \Delta t = |a_1.ts - a_2.ts|, \\ & \text{s.t.} \quad a_1.tid \neq a_2.tid, \\ & \qquad [a_1.addr, a_1.addr + a_1.size) \\ & \qquad\qquad \cap [a_2.addr, a_2.addr + a_2.size) \neq \emptyset, \\ & \qquad a_1.type = \mathsf{W} \vee a_2.type = \mathsf{W}, \\ & \qquad \Delta t \leq \tau. \end{aligned}$$

The first three constraints require the two accesses to form a conflicting access pair: they are performed by different threads, their accessed memory ranges overlap, and at least one access is a write. The last constraint is the temporal-admission constraint: the pair is retained as an MRP only when its temporal distance is no larger than the current threshold τ. The threshold τ is an admission-control parameter rather than a race-confirmation criterion; Section 3.3 describes how MRP-FUZZ adjusts it dynamically.

For feedback deduplication, MRP-FUZZ groups MRPs by their access-site pair $\langle n_1, n_2 \rangle$. MRPs with the same access-site pair but different stack-context pairs $\langle s_1, s_2 \rangle$ are treated as distinct context variants of that pair. For each access-site pair, MRP-FUZZ retains at most a configurable number of context variants.

3.2. Fuzzing Framework

Based on MRPs, we propose a novel kernel concurrency fuzzing framework, MRP-FUZZ, which decouples input generation and thread scheduling into two separate instances. Starting from a corpus of initial inputs, the input-generation instance generates and mutates inputs and executes them concurrently. This instance collects the new May Race Pairs from these executions. The corresponding inputs of the new MRPs are stored in the corpus for further mutation, and also in the schedule-worthy input queue for scheduling. The inputs in the schedule-worthy input queue are consumed by the thread-scheduling instance. Given an input, the thread-scheduling instance schedules its thread executions to force the conflicting access pair recorded in the corresponding MRP to occur concurrently. The success of such scheduling indicates a real data race has been detected. The scheduled syscall groups are dequeued from the schedule-worthy input queue. In this framework, the input generation and thread scheduling are connected only with the schedule-worthy inputs.

3.3. Dynamic Time-Threshold Control

As defined above, an observed conflicting access pair is admitted as an MRP only when the temporal distance between the two accesses is no larger than a threshold τ. In other words, τ acts as an admission-control parameter for the schedule-worthy input corpus. The value of τ creates a tradeoff between candidate supply and candidate selectivity. A smaller τ keeps only the inputs whose access pairs occur very close in time, making the retained inputs more selective and usually more promising for triggering data races under scheduling. However, an overly small threshold may produce too few schedule-worthy inputs and leave the scheduling instance underutilized. In contrast, a larger τ increases the candidate supply, but also admits inputs whose access pairs have larger temporal distances. Such pairs are less likely to be brought into overlap in time through scheduling and are more likely to include address-reuse artifacts, in which the same memory address is accessed after the underlying object has changed.

A fixed time threshold is unsuitable for our design. The value of the suitable time threshold may differ for different kernel modules. Even for the same kernel module, the value can change if underlying hardware is different. More critically, the fixed threshold cannot maintain this balance across different kernel modules or throughout a fuzzing campaign. The production speed of schedule-worthy inputs may change as input generation reaches different subsystem states, while the consumption speed may also vary when scheduling different inputs. Consequently, the fixed threshold may either starve the scheduling instance or cause excessive candidate accumulation.

To tackle this problem, we propose backpressure-guided dynamic threshold control, as shown in Algorithm 1. The design is inspired by adaptive admission control and backpressure in staged event-driven service systems, where requests flow through explicit queues between stages and upstream stages react to downstream overload signals [29]. Similarly, we model the input-generation instance as a producer, the thread-scheduling instance as a consumer, and the schedule-worthy input corpus as the queue connecting them. The threshold τ controls admission into this queue: increasing τ supplies more schedule-worthy inputs, whereas decreasing it admits fewer but more promising candidates.

At the end of each control interval T_c, the controller measures the number of newly retained schedule-worthy inputs P_k, scheduled inputs C_k, and pending inputs Q_k. To reduce the effect of short-term fluctuations, it maintains exponentially weighted moving averages:

$$\bar{P}_k = \rho \bar{P}_{k-1} + (1-\rho) P_k, \qquad \bar{C}_k = \rho \bar{C}_{k-1} + (1-\rho) C_k.$$

where $\rho \in [0, 1)$ is the smoothing factor. The controller estimates the buffered scheduling workload as

$$W_k = \frac{Q_k}{\max\left(\bar{C}_k, \epsilon\right)}.$$

where $\epsilon > 0$ prevents division by zero. In this paper, we set $T_c = 30\text{s}$, $W_{\text{low}} = 10$, $W_{\text{high}} = 40$, $\rho = 0.8$, $\epsilon = 1$, and use $\gamma_{\text{shrink}} = 0.5$ and $\Delta\tau = 0.05(\tau_{\max} - \tau_{\min})$ as the multiplicative tightening factor and additive relaxation step, respectively.

The controller maintains W_k between a low and a high watermark. When the workload is excessive and continues to grow, it multiplicatively decreases τ to rapidly tighten admission. When the corpus is empty, or the workload is low and continues to decrease, it additively increases τ to gradually admit more candidates. Otherwise, the threshold remains unchanged. The two watermarks avoid frequent oscillations, while the asymmetric adjustment limits candidate accumulation without starving the thread-scheduling instance.

3.4. Concurrent Execution Model

We use a syscall group as the input. A group is defined as the parallel composition of multiple syscall sequences, which are expected to execute concurrently:

$$G = S_0 \parallel S_1 \parallel \cdots \parallel S_{m-1},$$

where $\parallel$ denotes parallel composition, and each S_i is assigned to a distinct execution context. Each syscall sequence is a list of syscalls that execute on the system sequentially:

$$S_i = \langle c_{i,0}, c_{i,1}, \ldots, c_{i,n_i-1} \rangle,$$

where $c_{i,j}$ denotes the j-th syscall in S_i.

Algorithm 1 Backpressure-Guided Dynamic Threshold Control

Input: initial threshold τ_0, threshold bounds $\tau_{\min}$ and $\tau_{\max}$, control interval T_c, workload watermarks W_{low} and W_{high}, smoothing factor ρ, relaxation step Δ_τ, tightening factor γ_{shrink}
Output: dynamically adjusted threshold τ

$\tau \leftarrow \tau_0$; $\bar{P} \leftarrow 0$; $\bar{C} \leftarrow 0$;
while FUZZINGRUNNING() **do**
  WAIT(T_c);
  $P \leftarrow$ NEWMRPRECORDS()
  $C \leftarrow$ CONSUMEDMRPRECORDS()
  $Q \leftarrow$ PENDINGMRPRECORDS()
  $\bar{P} \leftarrow \rho\bar{P} + (1-\rho)P$
  $\bar{C} \leftarrow \rho\bar{C} + (1-\rho)C$
  $W \leftarrow Q / \max(\bar{C}, \epsilon)$
  if $W > W_{high} \wedge \bar{P} \geq \bar{C}$ **then**
    $\tau \leftarrow \max(\tau_{\min}, \gamma_{shrink}\tau)$
  else if $Q = 0 \vee (W < W_{low} \wedge \bar{P} \leq \bar{C})$ **then**
    $\tau \leftarrow \min(\tau_{\max}, \tau + \Delta_\tau)$
  SETTHRESHOLD(τ)
return τ;

Prior work observes that pairwise testing of program threads can substantially reduce testing complexity while preserving most bug-exposing capability [30]. Following this observation, we use two-member syscall groups by default in this paper.

In this paper, we execute a syscall group by running each syscall sequence of the group in a separate user-space process. We synchronize the release of these syscall sequences to ensure that these sequences start execution simultaneously.

4. Input Generation

The input generation of MRP-FUZZ aims to generate diverse syscall groups that expose a broad range of MRPs during concurrent execution, without involving thread scheduling. To achieve this goal, in addition to generating individually effective syscall sequences, the fuzzer needs to ensure that the syscall sequences in each group are semantically coordinated to access shared or related kernel state. Therefore, effective syscall-group mutation imposes semantic requirements, including jointly understanding the semantics of syscall sequences in each syscall group, identifying the kernel objects and subsystem states involved, and considering multiple syscall dependencies across the syscall group. To satisfy these semantic requirements and improve the diversity of the generated syscall groups, we propose LLM-assisted semantic mutation. By combining its reasoning capabilities with knowledge of system interfaces in prompts, including syscall semantics and kernel-resource lifecycle relations, an LLM can understand the semantics of the syscall group and mutate the two sequences in the group consistently, respecting module-specific syscall, resource, and environment constraints.

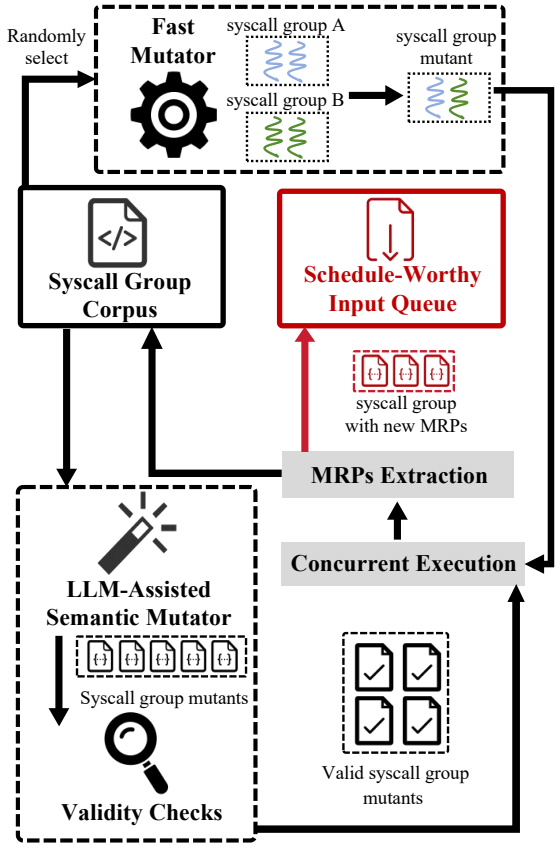

Figure 1. workflow of input generation.

4.1. LLM-Assisted Semantic Mutation

Workflow. Figure 1 shows the input-generation workflow. MRP-FUZZ maintains a syscall-group corpus and selects a seed syscall group from the corpus for further mutation. The LLM-assisted semantic mutator jointly rewrites the two syscall sequences in the seed group and generates syscall-group mutants. MRP-FUZZ validates each generated mutant using the built-in syzlang parser of Syzkaller. Each mutant that passes the check is treated as a valid syscall group and executed concurrently. Since LLM inference can introduce non-trivial latency, MRP-FUZZ issues LLM mutation requests asynchronously and submits completed LLM mutants to the validation stage when they become available, without blocking the input-generation pipeline.

To keep concurrent execution supplied while LLM requests are pending, MRP-FUZZ uses a fast mutator to generate syscall groups and execute them. When no completed LLM mutant is immediately available, MRP-FUZZ randomly selects two syscall groups from the corpus. The fast mutator generates an additional syscall group by randomly pairing syscall sequences sampled from the selected syscall groups. Each sequence in the group is further mutated by the lightweight mutations [31] of Syzkaller.

During concurrent execution, MRP-FUZZ runs each syscall sequence of a syscall group in a separate user-

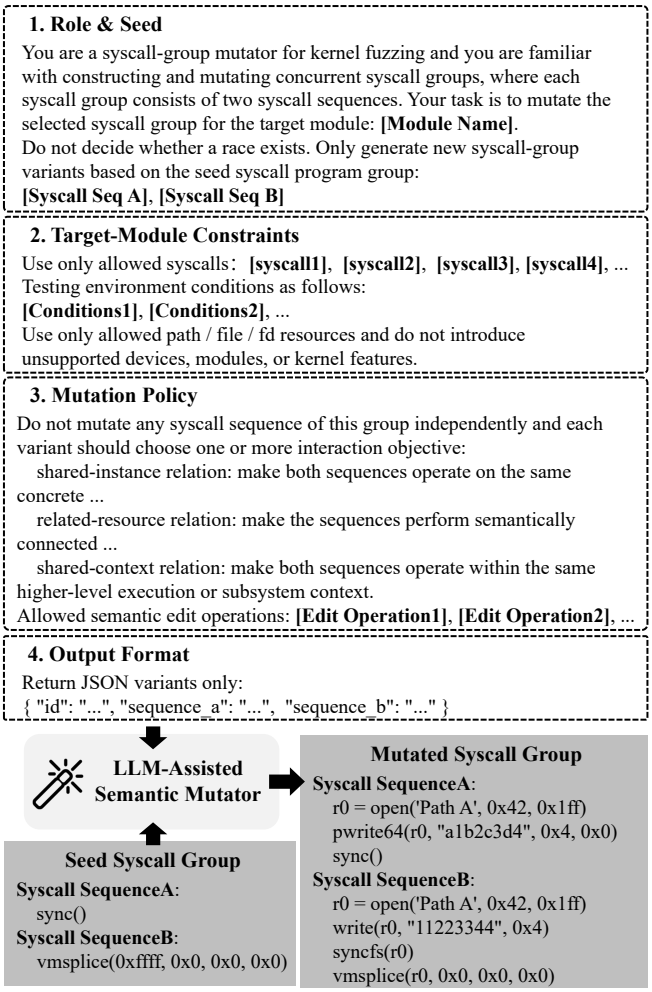

Figure 2. LLM-assisted semantic mutation.

space process and synchronizes their release, as described in Section 3.4. MRP-FUZZ then extracts MRPs from the resulting runtime memory-access trace. When an execution contributes one or more new MRPs, the corresponding syscall group is stored in the corpus for further mutation. MRP-FUZZ also creates a schedule-worthy input entry and adds it to the schedule-worthy input queue for thread scheduling discussed in Section 5.

Mutation. Figure 2 shows the structured prompt to the LLM for mutating the given syscall group. The prompt consists of four parts. The role & seed part defines the LLM as a syscall-group mutator and provides the selected seed syscall group. The target-module constraints specify the syscall whitelist, syntax requirements, testing environment, and resource-dependency rules. For each target module, we summarize these constraints from the actual testing environment, relevant Linux kernel source code and documentation, and Syzkaller's syscall and resource definitions. The mutation policy requires the LLM to treat the two sequences as a single mutation unit and coordinate their edits according to predefined cross-sequence interaction objectives. Finally, the output format restricts the response to a structured representation that can be parsed and validated automatically.

The mutation policy defines three complementary cross-sequence interaction objectives. Shared-instance relation specifies that both syscall sequences operate on the same concrete kernel object or resource instance. Related-resource relation coordinates operations on the same resource or on distinct but semantically related resources, such as a directory and one of its files, a link and its target, or different lifecycle stages of a resource. Shared-context relation places both syscall sequences in the same higher-level execution or subsystem context, such as a mount, namespace, device session, protocol state, or subsystem configuration. These objectives are not mutually exclusive, and a generated group may satisfy more than one of them.

The F2FS example in Figure 2 illustrates how semantic mutation strengthens cross-sequence interaction. The seed group loosely combines global synchronization with an operation on a file. The generated mutant adds file writes and synchronization operations around the same filesystem state, making the two sequences more likely to access overlapping filesystem state through local file operations and global synchronization.

Prioritized seed corpus. As in conventional fuzzing, mutation effort should be concentrated on interesting seeds rather than distributed uniformly across all seeds. It is especially important for LLM-assisted mutation, as its inference effort for mutating each seed is not trivial. MRP feedback is well-suited to this need because it captures both concrete evidence of cross-thread interaction and the novelty of that interaction. A previously unseen access-site pair represents a new pair of conflicting code locations and thus a new interaction target. For a covered access-site pair, its newly triggered stack contexts indicate a different calling path or kernel state, but its novelty decreases when more stack contexts of that pair have already been observed.

5. Thread Scheduling

Benefiting from MRPs, our thread scheduling is not performed blindly over the extremely large thread-interleaving space of all generated inputs. Instead, MRP-FUZZ concentrates its scheduling effort on the concrete conflicting access pairs specified in the corresponding MRPs of schedule-worthy syscall groups. MRP-FUZZ confirms each MRP as a real data race by attempting to schedule threads to force the conflicting accesses to occur concurrently.

As shown in Figure 3, we integrate three techniques to facilitate our scheduling: (1) candidate MRP prioritizing selects more promising syscall groups by excluding syscall groups with confirmed MRPs from further scheduling and deferring syscall groups with repeated unsuccessful scheduling attempts; (2) history-aware context recovery improves the reproducibility of conflicting access pairs by replaying all executed syscalls in a bounded history to reconstruct the required kernel resources, object relationships, and subsystem states; (3) to further improve reproducibility, context-

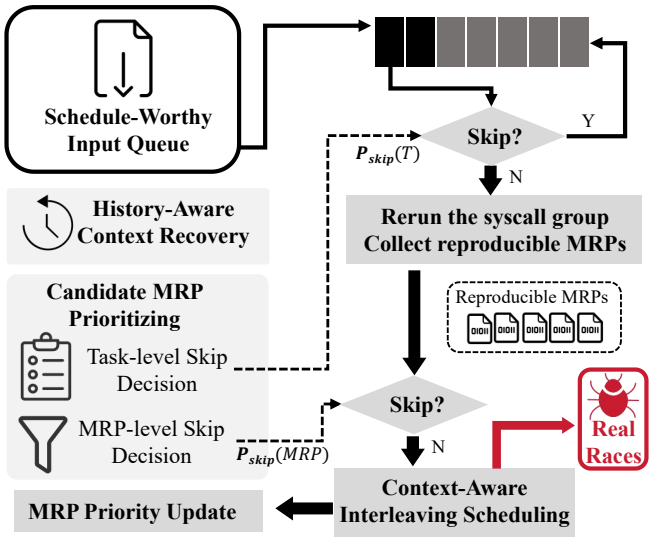

Figure 3. workflow of thread scheduling.

aware interleaving scheduling involves fine-grained thread controls according to the access site, stack context, and the hit counts of specific access sites.

Overall, the scheduling workflow proceeds as follows. MRP-FUZZ first dequeues a schedule-worthy input entry and applies candidate MRP prioritizing at the task level. If the task is skipped, it is deferred for future scheduling; otherwise, MRP-FUZZ replays its bounded history H and reruns the syscall group G to reproduce the MRPs associated with the entry. For each reproduced MRP, MRP-FUZZ further applies MRP-level prioritizing. The remaining MRPs are passed to context-aware interleaving scheduling, which uses the reproduced runtime context to locate the target access occurrences, applies a local delay, and updates the MRP priority according to the scheduling result.

5.1. Candidate MRP Prioritizing

During scheduling, each entry dequeued from the schedule-worthy input queue is represented as a scheduling task $T = \langle H, G, P_G \rangle$, where G is the schedule-worthy syscall group, H is its bounded history consisting of previously executed syscalls, and P_G is the set of MRPs associated with that entry. These MRPs serve as candidate scheduling targets: they indicate conflicting access pairs that may become real data races under targeted interleaving, but have not necessarily been confirmed yet. Candidate MRP prioritizing controls how MRP-FUZZ spends its limited scheduling budget. MRP-FUZZ maintains a skip probability for each candidate MRP and updates it according to previous scheduling results. A higher skip probability means that the candidate should be attempted less aggressively under the current budget, but it does not permanently remove the candidate from future scheduling; when a candidate is skipped, it is deferred and can be revisited subsequently.

Candidate MRP prioritizing is applied at two levels. At the MRP level, for each candidate MRP p, MRP-FUZZ maintains a scheduling state that records whether p has been confirmed and the number F_p of failed targeted-scheduling attempts. A candidate with no scheduling state receives $\Pr_{\text{skip}}(p) = 0$, so every newly discovered candidate receives an initial scheduling opportunity. An MRP confirmed as a real race receives $\Pr_{\text{skip}}(p) = 1$, so MRP-FUZZ avoids redundant scheduling. For the remaining candidates, MRP-FUZZ computes

$$\Pr_{\text{skip}}(p) = \min\left(P_{\max}, \frac{F_p + \alpha}{F_p + \alpha + \beta} \cdot (1 - \epsilon)\right),$$

where α and β are smoothing constants, ϵ preserves a minimum scheduling probability, and $P_{\max}$ caps the maximum skip probability.

At the task level, MRP-FUZZ uses the skip probabilities of the unresolved MRPs in P_G to decide whether to process a scheduling task $T = \langle H, G, P_G \rangle$. Let $U_G \subseteq P_G$ denote the set of candidate MRPs that have not yet been confirmed. MRP-FUZZ computes the task-level skip probability as their average skip probability:

$$\Pr_{\text{skip}}(T) = \frac{1}{|U_G|} \sum_{p_i \in U_G} \Pr_{\text{skip}}(p_i).$$

MRPs confirmed as real races are excluded from U_G. If $U_G = \emptyset$, the entry has no remaining scheduling target and is retired from the schedule-worthy input queue. Otherwise, MRP-FUZZ performs a probabilistic skip decision using $\Pr_{\text{skip}}(T)$. If the entry is skipped, it is appended to the tail of the schedule-worthy input queue rather than discarded. In this paper, we set $\alpha = 0.5$, $\beta = 0.5$, $\epsilon = 0.05$, and $P_{\max} = 0.90$.

5.2. History-Aware Context Recovery

Reproducing candidate MRPs may depend on kernel state created before the syscall group G itself. Such state includes resources, object references, filesystem state, device state, or subsystem state needed to reach the target variable access paths. Therefore, directly executing G from a clean state may fail to reproduce the MRPs in P_G. MRP-FUZZ addresses this problem by replaying a bounded execution history H before executing G. Formally, for a triggering syscall group $G_t = G$,

$$H = \langle G_{t-N_H}, G_{t-N_H+1}, \ldots, G_{t-1} \rangle$$

contains the N_H syscall groups executed immediately before G_t, in their original execution order. Replaying only these preceding groups avoids the cost and possible state perturbation of replaying the complete fuzzing history. The configurable bound N_H allows MRP-FUZZ to balance context recovery effectiveness and replay cost.

Thread scheduling uses this recovery step in both the collection run and the targeted scheduling run. Each run starts from a clean validation VM state, replays H, and then executes G, preventing different scheduling attempts

from interfering with each other. The collection run executes G without targeted delay and records, for each reproduced MRP, the minimum-gap runtime pair that matches the recorded access-site pair and stack-context pair in either access order within $\tau_{\max}$. The targeted scheduling run executes G with a targeted delay based on this metadata. Starting each run from a clean VM state would require repeatedly resetting the validation VM; Section 6 describes how MRP-FUZZ accelerates this process with VM snapshots.

5.3. Context-Aware Interleaving Scheduling

Context-aware interleaving scheduling is designed based on the observation that the same access site may be executed many times under the same stack context even within a single syscall execution. Delaying the wrong accesses of the access sites can miss the target MRP, while delaying all matching occurrences may perturb the original interleaving and drive the execution away from the target real race. Therefore, MRP-FUZZ uses finer-grained runtime context to distinguish different dynamic access occurrences of the same static access site and to locate the two critical occurrences corresponding to the MRP. Since these occurrences may drift across replays due to the randomness of concurrent execution, MRP-FUZZ does not rely on a single exact match. Instead, it first tries strict context matching and then gradually relaxes the matching condition with fallback strategies.

Let $P_G^r \subseteq P_G$ denote the set of MRPs reproduced during the collection phase. An access sequence number (ASN) is a per-thread counter that orders variable accesses within the same thread, allowing MRP-FUZZ to distinguish repeated dynamic occurrences of the same static access site. In the preceding collection phase, MRP-FUZZ selects, for each reproduced MRP, the matching runtime pair with the minimum temporal gap and records its access-site, stack-context, ASN and temporal-gap information.

Algorithm 2 shows the context-aware scheduling procedure. For each reproduced MRP, MRP-FUZZ first applies candidate MRP prioritizing at the MRP level. If the MRP is skipped, it is deferred for future attempts. Otherwise, MRP-FUZZ uses the collection metadata to locate the two target access occurrences during replay. It tries three matching strategies in order. ExactASN matches each access using its access site, stack context, and the exact ASN recorded during the collection run. If exact matching fails, BoundedASN retains the access-site and stack-context constraints while allowing the ASN to drift within a configured window around the recorded ASN. The window size is fixed by configuration. If both ExactASN and BoundedASN fail, StackOnly drops the ASN constraint and applies the scheduling action to every occurrence that matches the static access site and stack context.

After locating the two target accesses, MRP-FUZZ orders them according to their collection timestamps and denotes the earlier access as e_1. When an occurrence matching e_1 is reached during the targeted scheduling run, MRP-FUZZ arms a watchpoint-based observation window on the overlapping memory range and delays e_1 by $\lambda\Delta$, where Δ is the temporal gap recorded in the collection phase and λ is a configurable delay multiplier. If the opposite access performs a conflicting access to the overlapping range while this observation window remains active, MRP-FUZZ confirms the MRP as a real data race. Otherwise, the attempt is treated as an unsuccessful targeted-scheduling attempt and the MRP priority is updated accordingly.

Algorithm 2 Context-Aware Interleaving Scheduling

Input: validation task $T = \langle H, G, P_G \rangle$, re-observed candidate set P_G^r with collection metadata
Output: confirmed real race set R

$R \leftarrow \emptyset$
foreach $p \in P_G^r$ **do**
  if SKIPMRP(p) **then**
    DEFER(p)
    continue
  $(e_1, e_2) \leftarrow$ ORDEREDACCESSES(p)
  $\Delta \leftarrow$ TIMEGAP(p)
  $\mathcal{M} \leftarrow$ [ExactASN, BoundedASN, StackOnly]
  foreach $m \in \mathcal{M}$ **do**
    $\pi \leftarrow$ BUILDSCHEDULEPLAN($e_1, e_2, m, \lambda\Delta$)
    $res \leftarrow$ REPLAYANDSCHEDULE(H, G, π)
    if CONFIRMRACE(res, p) **then**
      $R \leftarrow R \cup \{(p, m, res)\}$
      UPDATEPRIORITY(p, res)
      break
    if TARGETMISS(res, p) **then**
      continue
    break
  if NOTCONFIRMED(p) **then**
    UPDATEPRIORITY(p, res)
return R

6. Implementation

We implement MRP-FUZZ across multiple software layers. First, the fuzzing and validation workflow is built on Syzkaller, with about 14.1 KLoC of new or modified Go/C++ code. Second, we implement the tracing and runtime support required by MRP extraction and context-aware interleaving scheduling, using an LLVM-based instrumentation pass and kernel runtime callbacks. During targeted validation, the runtime callback arms the watchpoint before the selected endpoint access and reports confirmation when the opposite endpoint accesses the overlapping memory range while the watchpoint remains active. This part contains about 3.3 KLoC of C/C++ code. Finally, we add snapshot-based reset support to Syzkaller's QEMU VM management interface to accelerate repeated candidate executions during validation, with about 1.0 KLoC of new or modified Go code. All code sizes are measured with cloc and exclude generated and third-party code.

Snapshot-accelerated validation. As discussed in Section 5.2, thread scheduling repeatedly starts collection and

TABLE 1. BASIC INFORMATION OF THE TESTED LINUX KERNEL MODULES.

Module	Description	Version	LOC
XFS	XFS file system	Linux 6.17	150K
Btrfs	Btrfs file system	Linux 6.17	112K
PTMX	PTMX/TTY subsystem	Linux 6.17	134K
OSS DSP	Sound HDA/core subsystem	Linux 6.17	104K
Bluetooth	Bluetooth protocol stack	Linux 6.17	50K
F2FS	Flash-Friendly File System	Linux 6.17	39K
JFS	Journaled File System	Linux 6.17	17K
Floppy	Floppy block device driver	Linux 6.17	4K

targeted scheduling runs from clean validation VM states, making VM reset a critical implementation cost. A straightforward implementation would reboot the VM and reinitialize the executor before every attempt, which introduces substantial fixed overhead. To reduce this cost, MRP-FUZZ saves a restorable base snapshot after the validation VM boots and the executor environment is initialized. Later validation attempts restore from this snapshot instead of performing a full reboot, replay the corresponding bounded history, and execute the candidate syscall group. This optimization reduces setup overhead without changing the replay history, scheduling strategy, or race-confirmation criterion.

7. Evaluation

7.1. Experiment Setup

We evaluate MRP-FUZZ on 8 common Linux kernel modules in Linux 6.17, including four file systems, three device-driver subsystems, and one network protocol stack. We select these modules because they are widely used and cover different parts of the kernel, allowing us to evaluate the effectiveness of our work across diverse kernel components. The information of these modules is listed in Table 1. We use CLOC [32] to count their lines of source code.

We run all experiments on a server with an AMD Ryzen Threadripper PRO 5975WX 32-Core processor and 128 GB of physical memory. For all experiments, we use **cset** to pin each experimental run to dedicated physical CPU cores and prevent CPU interference between concurrently running experiments. Unless otherwise stated, MRP-FUZZ evenly splits the allocated CPU cores between the input-generation instance and the thread-scheduling instance. The CPU-core allocation, memory budget, and execution time for each experiment are reported in the corresponding subsection.

7.2. Finding Real-World Data Races

During a three-week continuous testing campaign using DeepSeek-V4-Pro, MRP-FUZZ found 187 unique real-world data races after deduplication. Based on our manual analysis of their source code and triggering executions, we classify 59 races as harmful and the remaining 128 as benign. We reported all 59 harmful races to the corresponding kernel maintainers. At the time of writing, 39 have been confirmed by the maintainers, of which 15 have been fixed.

TABLE 2. SECURITY-IMPACT CLASSIFICATION OF THE 187 UNIQUE DATA RACES.

Module	Harmful / Benign	Memory Corruption / DoS	Data Corruption	Unexpected Behavior
XFS	5 / 17	1	4	0
Btrfs	2 / 19	0	2	0
PTMX	6 / 14	0	6	0
OSS DSP	11 / 2	0	5	6
Bluetooth	7 / 2	5	2	0
F2FS	8 / 33	1	6	1
JFS	11 / 19	0	10	1
Floppy	9 / 22	0	4	5
Total	59 / 128	7	39	13

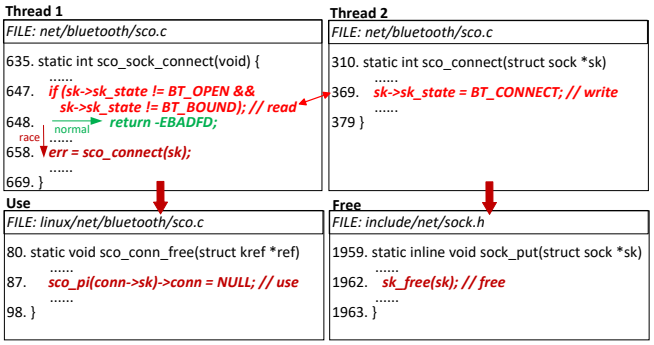

Figure 4. A harmful data race found in the Bluetooth stack.

7.3. Security Impact Analysis

Table 2 summarizes the security-impact classification of the detected data races. Among the 59 harmful races, 7 can cause memory corruption (*e.g.*, use-after-frees) or denial of service (DoS). 39 data races can corrupt critical kernel state or persistent data. Specifically, they change critical variables used in kernel state management, and the raced values can lead to inconsistent updates of file system state, device state, request state, or persistent metadata. 13 data races can cause unexpected behavior by affecting critical control-flow or state-machine decisions in the kernel modules. We present one representative example from each category.

Memory corruption. Figure 4 shows a harmful race in the Bluetooth SCO connect path. Here, `BT_OPEN`, `BT_BOUND`, and `BT_CONNECT` are Bluetooth socket states. Thread 1 checks the SCO socket state without holding the socket lock, while Thread 2 can concurrently update the same state in `sco_connect()`. The read in `sco_sock_connect()` and the write in `sco_connect()` therefore form a data race. In this case, Thread 1 can pass the check with the state

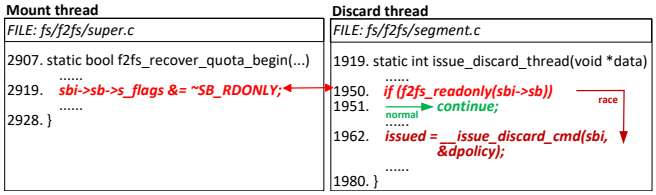

Figure 5. A harmful data race found in F2FS.

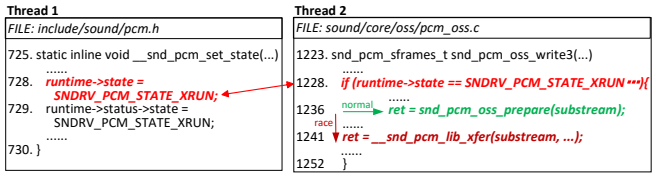

Figure 6. A harmful data race found in ALSA.

value `BT_OPEN` or `BT_BOUND`, and then Thread 2 assigns `BT_CONNECT` to the socket state. In the end, the two threads operate on the same socket with inconsistent connection ownership. As a result, the HCI disconnect handler for the connection installed by the earlier connect attempts may free the socket reference, while the later connect attempts in the other thread still accesses the socket through a stale SCO connection object whose associated socket has already been freed. This data race, therefore, triggers a use-after-free in the Bluetooth subsystem, which can corrupt kernel memory or crash the whole kernel.

Data Corruption. Figure 5 shows a harmful race in F2FS. During recovery of a read-only mount, the mount thread temporarily clears a flag `SB_RDONLY` to update recovery metadata, while the discard thread checks the same flag to decide whether discard I/O is allowed. These two accesses can occur concurrently and result in a data race. If the mount thread clears the flag first, the discard thread will observe the temporary flag value, which is not `SB_RDONLY`. In this case, the discard thread issues the discard or zone-reset requests before the mount thread restores the read-only flag. Such a data race makes a recovery-only state transition escape to the block device and discard ranges that still contain file-system data or metadata. As a result, it causes persistent data corruption.

Unexpected behavior. Figure 6 shows a data race in ALSA OSS PCM playback. In Thread 1, the IRQ-driven XRUN handling code updates `runtime->state` to `XRUN`. In Thread 2, the OSS write code reads `runtime->state` to decide whether recovery is required before transferring samples. Such two accesses can occur concurrently and cause a data race. If Thread 2 reads a stale `RUNNING` value from `runtime->state` first before `runtime->state` is assigned `XRUN` in Thread 1, Thread 2 can skip the required XRUN recovery and enter the transfer path while the stream is in the XRUN state. Consequently, the write operation proceeds without recovering the stream and fails instead of restoring normal playback.

TABLE 3. CONFIRMED REAL DATA RACES UNDER DIFFERENT INPUT-MUTATION STRATEGIES.

Module	GPT-5.4	DeepSeek-V4-Pro	Random
XFS	6	8	4
F2FS	22	17	7
Btrfs	11	11	9
PTMX	13	16	8
JFS	6	5	3
Floppy	5	10	3
DSP	4	5	2
Bluetooth	9	7	5
Total	76	79	41

7.4. Ablation Study on Input Generation

Experiment setup. This experiment evaluates whether LLM-assisted semantic mutation improves input generation and ultimately increases data race detection effectiveness. We compare two LLMs, GPT-5.4 and DeepSeek-V4-Pro, with random mutation. The random mutation baseline involves no LLM and uses only our fast mutator described in Section 4, which samples syscall sequences from the same syscall-group corpus, randomly mutate each sequence with syz-mutate, and then forms syscall groups. The three configurations use the same complete MRP-FUZZ framework and differ only in their input mutation strategy. For each kernel module, we run each configuration for 24 hours using four dedicated physical CPU cores and a 16 GB memory budget. In this experiment, we record the growth of MRPs collected under each configuration.

Analysis on LLM-assisted semantic mutation. Figure 7 reports the resulting MRP growth curves. Both LLM-assisted configurations generally produce more MRPs than random mutation, with particularly clear improvements on the file systems. This result indicates that LLM-assisted semantic mutation generates syscall groups that can trigger more MRPs, as the syscalls in these groups are more likely to access shared or related kernel state.

More MRPs are useful only if they result in more confirmed races. Table 3 shows that the GPT-5.4 and DeepSeek-V4-Pro configurations confirm 76 and 79 races, respectively, compared with 41 for random mutation. This corresponds to improvements of 85.4% and 92.7%. Both LLM-assisted configurations outperform random mutation on each evaluated kernel module. The similar gains obtained with two different LLMs suggest that our semantic mutation can work well with different LLMs.

7.5. Ablation Study on Thread Scheduling

Experiment setup. This experiment evaluates the contributions of each design component in the thread-scheduling

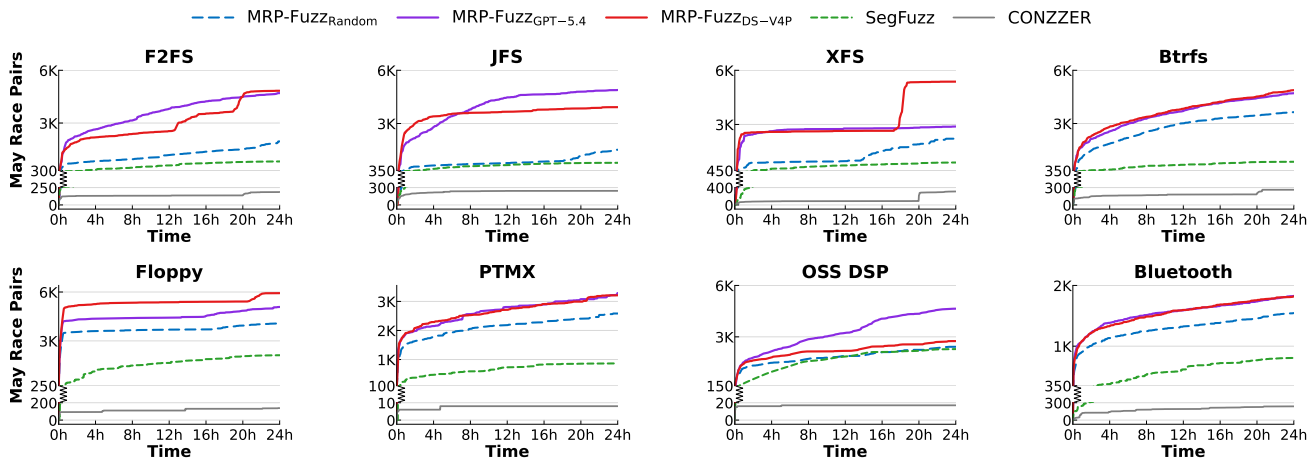

Figure 7. Growth of MRPs produced by different MRP-FUZZ mutation strategies and prior kernel concurrency fuzzers.

instance of MRP-FUZZ. Unlike input generation, thread scheduling does not aim to generate more May Race Pairs. Instead, it aims to efficiently schedule the thread execution of the target syscall groups, forcing the conflicting access pair specified in the MRP to occur concurrently.

To isolate the effects of thread scheduling from the randomness of input generation, all experiments in this section use a corpus of syscall groups with MRPs for each module. The corpus is generated by the DeepSeek-V4-Pro configuration described in Section 7.4. For each module, all thread scheduling configurations consume the same corpus and use the same scheduling budget, differing only in the enabled design components. For each kernel module, we run each configuration for 24 hours using two dedicated physical CPU cores and an 8 GB memory budget.

We compare the full configuration with three ablated configurations. NO-HISTORY disables history-aware context recovery and executes each candidate group without replaying its saved execution history. NO-ASN disables ASN-based occurrence matching and uses only STACKONLY, retaining access-site and stack-context constraints but applying delay to every matching dynamic occurrence. NO-BACKOFF disables probabilistic candidate skipping and deferral based on previous scheduling outcomes.

Table 4 reports the number of real data races. Table 5 reports scheduling-budget utilization for the full, No-ASN, and No-backoff configurations. In Table 5, a processed pair is either scheduled to trigger potential data races or probabilistically skipped and deferred by candidate MRP prioritizing. Accordingly, the number of executed scheduling attempts is equal to the number of processed pairs excluding the number of skips.

Analysis on history-aware context recovery. As shown in Table 4, history-aware context recovery substantially improves candidate reproducibility. The full configuration confirms 83 real data races, whereas No-history confirms only 35 real data races, indicating a reduction of 57.8%.

TABLE 4. EFFECT OF THREAD-SCHEDULING COMPONENTS ON CONFIRMED REAL DATA RACES.

Module	Full	No-history	No-ASN	No-backoff
XFS	7	2	2	3
Btrfs	12	5	7	5
PTMX	18	8	9	13
OSS DSP	6	3	5	0
Bluetooth	6	5	4	4
F2FS	18	8	11	7
JFS	5	1	2	2
Floppy	11	3	6	6
Total	83	35	46	40

The effectiveness degradation appears across all evaluated modules. Without recent nearby execution history, a candidate group may fail to reconstruct the file system, device, resource-reference, or protocol state under which its target conflicting access pairs were originally observed. The data races still confirmed by No-history indicate that some syscall groups are self-contained, but the large reduction shows that many candidate MRPs depend on state established by preceding executions.

Analysis on context-aware interleaving scheduling. Context-aware interleaving scheduling substantially improves target-localization precision. No-ASN confirms 46 real data races, compared with 83 for the full configuration, indicating a reduction of 44.6%. This effectiveness degradation is not caused by fewer scheduling opportunities. No-ASN executes 4,564 targeted scheduling attempts and processes 13,318 candidate pairs, compared with 4,261 attempts and 12,485 processed pairs for the full configuration. Without ASN information, multiple dynamic occurrences sharing the same access site and stack context may be treated as the target occurrence. Applying delay to these additional

TABLE 5. SCHEDULING-BUDGET UTILIZATION BY MODULE.

Module	Full Skips	Full Processed	No-ASN Skips	No-ASN Processed	No-backoff Processed
XFS	297	712	511	1,008	407
Btrfs	1,549	2,103	1,528	2,146	504
PTMX	462	1,013	427	979	546
OSS DSP	706	971	835	1,123	276
Bluetooth	220	450	202	450	236
F2FS	2,264	3,064	2,225	3,046	774
JFS	463	950	511	1,001	489
Floppy	2,263	3,222	2,515	3,565	967
Total	8,224	12,485	8,754	13,318	4,199

occurrences perturbs the execution without reliably bringing the intended MRP endpoints into overlap, reducing the effectiveness of targeted interleaving exploration.

Analysis on candidate MRP prioritizing. Candidate MRP prioritizing improves the allocation of the scheduling budget. No-backoff confirms 40 real data races, compared with 83 for the full configuration, indicating a reduction of 51.8%. The two configurations execute a similar number of targeted scheduling attempts: 4,199 for No-backoff and 4,261 for the full configuration. However, the full configuration additionally skips or defers 8,224 low-priority candidates and consequently advances over 12,485 candidate pairs, approximately 2.97 times the 4,199 pairs reached by No-backoff. Without backoff, low-potential candidates continue to consume scheduling attempts and prevent interleaving exploration from reaching a broader portion of the MRP corpus.

Overall, the three mechanisms address different challenges in thread scheduling for confirming data races. History-aware context recovery reconstructs the state required to reproduce candidate accesses, ASN-aware matching anchors scheduling delay to the intended dynamic occurrences, and candidate MRP prioritizing prevents low-potential candidates from repeatedly monopolizing the scheduling budget. Together, these mechanisms enable MRP-FUZZ to confirm substantially more MRP candidates into real data races with efficient thread scheduling under a bounded scheduling budget.

7.6. Comparison with prior approaches

Experiment setup. We compare MRP-FUZZ with two state-of-the-art concurrency-aware kernel fuzzers, SegFuzz and CONZZER. We examine two complementary outcomes from the same 24-hour end-to-end runs: the growth of MRPs during execution and the number of real data races confirmed by the end of the run.

For MRP-FUZZ, we reuse the three configurations evaluated in Section 7.4: $\mathrm{MRP\text{-}Fuzz}_{\mathrm{GPT\text{-}5.4}}$, which uses GPT-5.4-assisted semantic mutation; $\mathrm{MRP\text{-}Fuzz}_{\mathrm{DS\text{-}V4P}}$, which uses DeepSeek-V4-Pro-assisted semantic mutation; and $\mathrm{MRP\text{-}Fuzz}_{\mathrm{Random}}$, which uses random syscall-group mutation. In all three configurations of MRP-FUZZ, input generation and thread-interleaving exploration run concurrently, and the dynamic time-threshold controller remains enabled. SegFuzz and CONZZER run in their original modes. All approaches use the same evaluation setup, *i.e.*, four physical CPU cores, 16 GB memory, and 24-hour testing per kernel module.

We extract MRPs from the executions of SegFuzz and CONZZER using the MRP construction and accounting rules described in Section 3.1. Because these baselines do not implement MRP-FUZZ's dynamic threshold controller, we use $\tau_{\max}$, the maximum threshold permitted by the controller, when extracting their MRPs. This permissive setting admits conflicting access pairs with larger temporal distances and therefore avoids artificially limiting the number of MRPs attributed to the baselines.

MRP analysis. Figure 7 shows the growth of MRPs during runs. The MRP-FUZZ curves are the same results analyzed in Section 7.4 and are included here for direct comparison with SegFuzz and CONZZER. The three MRP-FUZZ configurations generally produce more MRPs across the evaluated kernel modules, with clear advantages on the file systems. Importantly, $\mathrm{MRP\text{-}Fuzz}_{\mathrm{Random}}$ produces more MRPs than the two baselines. Since this configuration does not use LLM-assisted mutation, this result demonstrates the benefit of decoupling input generation and thread scheduling through the MRP interface, which significantly improves the fuzzing efficiency.

Data race detection. Table 6 reports the confirmed real data races from the same end-to-end runs. The MRP-FUZZ rows reuse the results reported in Table 3 and are repeated here to enable a direct per-module comparison with the baselines. $\mathrm{MRP\text{-}Fuzz}_{\mathrm{DS\text{-}V4P}}$ confirms 79 real data races, compared with 19 for SegFuzz and 11 for CONZZER. This corresponds to improvements of 4.16× and 7.18×, respectively. Even $\mathrm{MRP\text{-}Fuzz}_{\mathrm{Random}}$ confirms 41 real data races, which is 2.16× the result of SegFuzz and 3.73× that of CONZZER. This result shows that the improvement does not rely solely on LLM-assisted mutation: decoupling input generation from thread scheduling already provides a substantial advantage over the baselines that couple these two phases.

TABLE 6. CONFIRMED REAL DATA RACES IN THE COMPARISON WITH PRIOR APPROACHES.

Module	MRP-FUZZ			SegFuzz	CONZZER
	GPT-5.4	DS-V4P	Random		
XFS	6	8	4	3	1
F2FS	22	17	7	6	3
Btrfs	11	11	9	2	1
PTMX	13	16	8	1	1
JFS	6	5	3	3	4
Floppy	5	10	3	2	1
OSS DSP	4	5	2	1	0
Bluetooth	9	7	5	1	0
Total	76	79	41	19	11

Note: DS-V4P denotes DeepSeek-V4-Pro.

Overall, these results show that MRP-FUZZ improves both the supply of concrete scheduling targets (*i.e.*, MRPs) and their conversion into real data races. The decoupled framework provides significant improvement over existing approaches.

8. Related Work

8.1. Coverage-Guided Fuzzing

Coverage-guided fuzzing, like AFL, uses execution coverage to collect interesting inputs for further mutation [33]. Later work improves this paradigm through better seed scheduling and search strategies [34]–[36], or through more informative feedback and structure-aware mutation [8]–[11]. These techniques are effective at increasing code coverage and exposing diverse sequential execution paths. However, such feedback mainly captures sequential path information and cannot indicate whether an input has the potential to trigger interesting concurrent executions. In contrast, MRPs capture conflicting variable-access pairs that occur within a short time interval, providing direct guidance for focusing computing resources on syscall groups that are more likely to expose data races.

8.2. Kernel and Concurrency Fuzzing

Coverage-guided fuzzing has been widely applied to kernel testing, where inputs are typically represented as syscall programs. Syzkaller and kAFL establish the basic coverage-guided kernel fuzzing framework [12], [37]. Subsequent systems improve kernel input generation by better modeling syscall semantics, exploring complex kernel states, and supporting diverse kernel interfaces and subsystems [13]–[17], [38]–[48]. These techniques substantially broaden the kernel code paths exercised by generated inputs, but they still focus solely on exploring the input space without considering thread interleavings. As a result, they may miss data races that manifest only under rare or specific interleavings.

Existing kernel concurrency fuzzing additionally explores the thread-interleaving space. Prior work improves this thread scheduling through search space reduction, concurrency-aware feedback, and controlled interleaving scheduling [20]–[22], [26]–[28], [49]–[53]. However, most existing approaches still couple input generation with interleaving exploration, causing substantial testing effort to be spent scheduling inputs with little potential to trigger real data races. In contrast, MRP-FUZZ decouples these two searches: its input-generation stage collects MRPs during concurrent execution, while its interleaving-exploration stage uses the collected MRPs to concentrate interleaving exploration on promising regions of the interleaving space.

8.3. LLM-Assisted Testing

LLMs have been increasingly applied to software testing [54]–[56], and various fuzzing tasks such as input generation, mutation, and fuzzer synthesis [57]–[63]. In kernel fuzzing, KernelGPT uses LLMs to synthesize and refine Syzkaller syscall specifications, while SyzAgent uses them to guide test-case generation and mutation during kernel fuzzing [41], [64]. Themis uses LLMs to generate RPC-based tests for statically identified distributed race candidates, followed by directed fuzzing of their input parameters [65]. In contrast, MRP-FUZZ uses LLMs to semantically mutate existing concurrent syscall groups while preserving and strengthening their concurrent interactions. To our knowledge, MRP-FUZZ is the first kernel concurrency fuzzer to use LLMs for semantic mutation of concurrent syscall groups.

9. Conclusion

This paper presents MRP-FUZZ, a kernel concurrency fuzzing framework that decouples input generation and thread scheduling through May Race Pairs (MRPs). By using MRPs to guide semantic mutation during input generation and to focus subsequent scheduling effort on promising interleavings, MRP-FUZZ avoids spending substantial testing effort on inputs with little potential to trigger data races. Our evaluation on eight Linux kernel modules found 187 real data races, including 59 harmful data races, 39 of which have been confirmed by kernel maintainers and 15 of which have been fixed. Under the same testing budget, MRP-FUZZ detects substantially more data races than state-of-the-art kernel concurrency fuzzers.

More broadly, MRP-FUZZ shows that lightweight data race candidate discovery can be decoupled from expensive thread scheduling through an explicit runtime abstraction. We instantiate this abstraction as May Race Pair, but other forms of concurrency-related abstraction can play the same role, as long as they are inexpensive to collect yet informative enough to guide thread scheduling and race confirmation. We believe our decoupling design offers a general and effective paradigm for concurrency fuzzing beyond the specific design of MRP-FUZZ.

References

[1] CVE Program, "CVE-2022-49171," 2025, published 26 February 2025; accessed 10 June 2026. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2022-49171

[2] ——, "CVE-2022-49554," 2025, published 26 February 2025; accessed 10 June 2026. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2022-49554

[3] ——, "CVE-2022-50145," 2025, published 18 June 2025; accessed 10 June 2026. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2022-50145

[4] ——, "CVE-2025-40318," 2025, published 8 December 2025; accessed 10 June 2026. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2025-40318

[5] ——, "CVE-2021-4202," 2022, published 25 March 2022; accessed 10 June 2026. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2021-4202

[6] ——, "CVE-2023-0266," 2023, published 30 January 2023; accessed 10 June 2026. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2023-0266

[7] ——, "CVE-2016-5195 (Dirty COW)," 2016, accessed 10 June 2026. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2016-5195

[8] P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *2018 IEEE Symposium on Security and Privacy (S&P)*, 2018.

[9] C. Lemieux and K. Sen, "FairFuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2018.

[10] V.-T. Pham, M. Böhme, A. E. Santosa, A. R. Caciulescu, and A. Roychoudhury, "Smart greybox fuzzing," *IEEE Transactions on Software Engineering*, vol. 47, no. 9, pp. 1980–1997, 2021.

[11] J. Wang, B. Chen, L. Wei, and Y. Liu, "Superion: Grammar-aware greybox fuzzing," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019.

[12] Google syzkaller project, "syzkaller: Kernel fuzzer," software repository; accessed 10 June 2026. [Online]. Available: https://github.com/google/syzkaller

[13] S. Pailoor, A. Aday, and S. Jana, "MoonShine: Optimizing OS fuzzer seed selection with trace distillation," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.

[14] J. Corina, A. Machiry, C. Salls, Y. Shoshitaishvili, S. Hao, C. Kruegel, and G. Vigna, "DIFUZE: Interface aware fuzzing for kernel drivers," in *Proceedings of the 24th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017.

[15] S. M. S. Talebi, H. Tavakoli, H. Zhang, Z. Zhang, A. A. Sani, and Z. Qian, "Charm: Facilitating dynamic analysis of device drivers of mobile systems," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.

[16] W. Xu, H. Moon, S. Kashyap, P.-N. Tseng, and T. Kim, "Fuzzing file systems via two-dimensional input space exploration," in *Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P)*, 2019.

[17] S. Kim, M. Xu, S. Kashyap, J. Yoon, W. Xu, and T. Kim, "Finding semantic bugs in file systems with an extensible fuzzing framework," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[18] Linux Kernel Documentation, "Kernel concurrency sanitizer (KCSAN)," accessed 10 June 2026. [Online]. Available: https://docs.kernel.org/dev-tools/kcsan.html

[19] Google Kernel Sanitizers Project, "Kernel thread sanitizer (KTSAN)," accessed 10 June 2026. [Online]. Available: https://google.github.io/kernel-sanitizers/KTSAN.html

[20] D. R. Jeong, K. Kim, B. Shivakumar, B. Lee, and I. Shin, "RAZZER: Finding kernel race bugs through fuzzing," in *Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P)*, 2019.

[21] S. Gong, D. Altınbüken, P. Fonseca, and P. Maniatis, "Snowboard: Finding kernel concurrency bugs through systematic inter-thread communication analysis," in *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021.

[22] S. Gong, D. Peng, D. Altınbüken, P. Fonseca, and P. Maniatis, "Snowcat: Efficient kernel concurrency testing using a learned coverage predictor," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[23] H. Chen, S. Guo, Y. Xue, Y. Sui, C. Zhang, Y. Li, H. Wang, and Y. Liu, "MUZZ: Thread-aware grey-box fuzzing for effective bug hunting in multithreaded programs," in *Proceedings of the 29th USENIX Security Symposium*, 2020.

[24] N. Vinesh and M. Sethumadhavan, "ConFuzz—a concurrency fuzzer," in *First International Conference on Sustainable Technologies for Computational Intelligence*, vol. 1045, 2020.

[25] C. Liu, D. Zou, P. Luo, B. B. Zhu, and H. Jin, "A heuristic framework to detect concurrency vulnerabilities," in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018.

[26] Z.-M. Jiang, J.-J. Bai, K. Lu, and S.-M. Hu, "CONZZER: Context-sensitive and directional concurrency fuzzing for data-race detection," in *Proceedings of the 29th Annual Network and Distributed System Security Symposium (NDSS)*, 2022.

[27] D. R. Jeong, B. Lee, I. Shin, and Y. Kwon, "SEGFUZZ: Segmentizing thread interleaving to discover kernel concurrency bugs through fuzzing," in *Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P)*, 2023.

[28] J. Xu, D. Wolff, X. Y. Han, J. Li, and A. Roychoudhury, "Concurrency fuzzing of the linux kernel with eBPF," in *Proceedings of the 35th USENIX Security Symposium (USENIX Security 26)*, 2026, to appear.

[29] M. Welsh and D. Culler, "Adaptive overload control for busy internet servers," in *4th USENIX Symposium on Internet Technologies and Systems (USITS 03)*. Seattle, WA: USENIX Association, Mar. 2003.

[30] S. Lu, S. Park, E. Seo, and Y. Zhou, "Learning from mistakes: A comprehensive study on real world concurrency bug characteristics," in *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2008.

[31] Syzkaller Project Authors, "`syz-mutate`: Syzkaller program mutation tool," software repository; accessed 12 June 2026. [Online]. Available: https://github.com/google/syzkaller/tree/master/tools/syz-mutate

[32] A. Danial, "CLOC: Count lines of code," software repository; accessed 10 June 2026. [Online]. Available: https://github.com/AlDanial/cloc

[33] M. Zalewski, "American Fuzzy Lop (AFL)," software project; accessed 10 June 2026. [Online]. Available: https://lcamtuf.coredump.cx/afl/

[34] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain," in *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.

[35] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proceedings of the 24th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017.

[36] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, 2020.

[37] S. Schumilo, C. Aschermann, R. Gawlik, S. Schinzel, and T. Holz, "kAFL: Hardware-assisted feedback fuzzing for OS kernels," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.

[38] W. Chen, Y. Wang, Z. Zhang, and Z. Qian, "SyzGen: Automated generation of syscall specification of closed-source macOS drivers," in *Proceedings of the 28th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.

[39] Y. Hao, G. Li, X. Zou, W. Chen, S. Zhu, Z. Qian, and A. A. Sani, "SyzDescribe: Principled, automated, static generation of syscall descriptions for kernel drivers," in *Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P)*, 2023.

[40] W. Chen, Y. Hao, Z. Zhang, X. Zou, D. Kirat, S. Mishra, D. Schales, J. Jang, and Z. Qian, "SyzGen++: Dependency inference for augmenting kernel driver fuzzing," in *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P)*, 2024.

[41] C. Yang, Z. Zhao, and L. Zhang, "KernelGPT: Enhanced kernel fuzzing via large language models," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[42] B. Zhao, Z. Li, S. Qin, Z. Ma, M. Yuan, W. Zhu, Z. Tian, and C. Zhang, "StateFuzz: System call-based state-aware linux driver fuzzing," in *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*, 2022.

[43] X. Tan, Y. Zhang, J. Lu, X. Xiong, Z. Liu, and M. Yang, "SyzDirect: Directed greybox fuzzing for linux kernel," in *Proceedings of the 30th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.

[44] M. Fleischer, D. Das, P. Bose, W. Bai, K. Lu, M. Payer, C. Kruegel, and G. Vigna, "ACTOR: Action-guided kernel fuzzing," in *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*, 2023.

[45] H. Peng and M. Payer, "USBFuzz: A framework for fuzzing USB drivers by device emulation," in *Proceedings of the 29th USENIX Security Symposium (USENIX Security 20)*, 2020.

[46] D. Song, F. Hetzelt, D. Das, C. Spensky, Y. Na, S. Volckaert, G. Vigna, C. Kruegel, J.-P. Seifert, and M. Franz, "PeriScope: An effective probing and fuzzing framework for the hardware-OS boundary," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2019.

[47] Z. Ma, B. Zhao, L. Ren, Z. Li, S. Ma, X. Luo, and C. Zhang, "PrIntFuzz: Fuzzing linux drivers via automated virtual device simulation," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022.

[48] D. Song, F. Hetzelt, J. Kim, B. B. Kang, J.-P. Seifert, and M. Franz, "Agamotto: Accelerating kernel driver fuzzing with lightweight virtual machine checkpoints," in *Proceedings of the 29th USENIX Security Symposium (USENIX Security 20)*, 2020.

[49] M. Musuvathi, S. Qadeer, T. Ball, G. Basler, P. A. Nainar, and I. Neamtiu, "Finding and reproducing heisenbugs in concurrent programs," in *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2008.

[50] P. Fonseca, R. Rodrigues, and B. B. Brandenburg, "SKI: Exposing kernel concurrency bugs through systematic schedule exploration," in *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014.

[51] J. Erickson, M. Musuvathi, S. Burckhardt, and K. Olynyk, "Effective data-race detection for the kernel," in *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010.

[52] M. Xu, S. Kashyap, H. Zhao, and T. Kim, "KRACE: Data race fuzzing for kernel file systems," in *Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P)*, 2020.

[53] D. Wolff, Z. Shi, G. J. Duck, U. Mathur, and A. Roychoudhury, "Greybox fuzzing for concurrency testing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, 2024.

[54] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

[55] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "ChatUniTest: A framework for LLM-based test generation," in *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, Companion Volume (FSE Companion)*, 2024.

[56] N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, "Automated unit test improvement using large language models at meta," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion '24)*, 2024.

[57] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.

[58] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.

[59] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023.

[60] H. Xu, W. Ma, T. Zhou, Y. Zhao, K. Chen, Q. Hu, Y. Liu, and H. Wang, "CKGFuzzer: LLM-based fuzz driver generation enhanced by code knowledge graph," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2025, pp. 243–254.

[61] C. Chen, B. Dolan-Gavitt, and Z. Lin, "ELFuzz: Efficient input generation via LLM-Driven synthesis over fuzzer space," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, Aug. 2025.

[62] K. Zhang, Z. Li, D. Wu, S. Wang, and X. Xia, "Low-cost and comprehensive non-textual input fuzzing with LLM-Synthesized input generators," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, Aug. 2025.

[63] Z. Jia, X. Yin, S. Gan, C. Zhang, H. Liu, J. Ji, E. Song, R. Cai, J. Tan, and S. Liu, "PANGOLIN: Fuzzing multilingual IoT firmware with LLM-Driven code analysis," in *35th USENIX Security Symposium (USENIX Security 26)*, 2026, to appear.

[64] X. Li, Z. Yuan, Z. Zhang, Y. Sun, and L. Zhang, "Towards large language model guided kernel direct fuzzing," in *Proceedings of the International Conference on Fundamental Approaches to Software Engineering (FASE)*, 2025, pp. 33–42.

[65] H. Cao, J. He, T. Dai, and G. Jin, "Themis: Detecting distributed concurrency bugs through RPC-driven race-directed test generation and fuzzing," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, May 2026.